Racing Line Steering from Raw LiDAR on an Arduino Nano 33 BLE Sense Rev2

Final Project Report — Team #2
EE 446: TinyML for Ultra Low-Power Edge Computing, Summer 2026

Sam Kutsyn (2581500) Volodymyr Kuchera (2523181)

21 August 2026

Abstract

A 1,146-parameter network reads a 28-beam 2D LiDAR scan and steers a simulated race car at 50 Hz; quantized to int8 it fits in 2,412 bytes of Arduino Nano 33 BLE Sense Rev2 flash, runs in 138 µs, and completes a lap on 14 of 16 layouts that model selection never saw. Code, artifacts and build instructions: https://github.com/v-kut/tinyml

1 Team Member Contributions

1.1 Sam Kutsyn

Sam wrote all the code and ran every experiment. That means the simulator (track generator, wall geometry, ray caster, car model, LiDAR fan and the pure-pursuit expert that supplies the labels), the training pipeline (the Gymnasium environment, the reward, and the three stages: clone, PPO, distill), and the whole deployment path (the int8 quantizer, the C code generator, the `tinyml.h` kernel on the board and the USB link on both ends). He also wrote the test suite, the live viewer, the findings notes, the slides and this report, and did the debugging behind all of it.

1.2 Volodymyr Kuchera

Volodymyr reviewed the work and made the calls; he wrote no code. He decided to drop the proposal's imitation-only plan once the clone turned out to crash on every track, to judge models by laps driven instead of validation error, and to spend flash on the exact tanh table rather than save 22 µs with a cheaper activation. He also turned down full-integer requantization, which is what keeps the board's output identical to the host emulator.

2 Introduction and Project Objective

2.1 Problem

The target is a 1/28 Kyosho Mini-Z retrofit. There is no room for a compute board, no power budget for a GPU, and the control deadline is set by how fast the car is moving. Full-scale autonomy solves this with SLAM on heavy compute, which does not fit at this scale, so the question is whether a network small enough to flash can drive from raw range data alone. It is a good TinyML problem: the sensor output is small (a 1-D vector, so no convolutional stack), the task is limited by latency rather than throughput, and the deadline is hard — 20 ms per loop at 50 Hz, which no cloud round trip can meet.

2.2 Task

Closed-loop nonlinear regression. The input is 61 floats: the normalized LiDAR sweep, its first difference, four car-state channels, and the last throttle command. The output is two floats, steering and a signed throttle/brake pedal, both in $[-1, 1]$. Each output changes the next input, so errors build up step by step.

2.3 Deployment setting

Arduino Nano 33 BLE Sense Rev2 (nRF52840, Cortex-M4F at 64 MHz, 1 MB flash, 256 kB RAM). There is no physical LiDAR this quarter, so the host runs the simulator and sends the 61 floats over USB-CDC; the board replies with the two commands and its own timing in microseconds. Every action the car obeys is computed on the MCU.

2.4 Success criteria

Table 1: Success criteria against both evaluation sets. The selection set is the eight layouts that also chose `best.pt`; the test set is 16 layouts selection never saw (Section 6.4). The task row passes on the first and fails the no-crash clause on the second, which is the more honest number.

criterion	target	on selection set	on test set
task	one lap, no crashes, within 5 % of float32 reward	1.740 laps, 0/8 crashes, −0.60 %	1.480 laps, 1/16 crashes, −4.5 %
footprint	under 4 kB flash, under 1 kB RAM	2,412 B, 549 B RAM	same model
latency	under 2 ms, 10 % of the 20 ms step	138 µs, 0.7 %	same kernel
correctness	board output identical to host	max 5.96×10^{-8}	same

3 Dataset and Experimental Setup

3.1 Source

With no LiDAR hardware and no RC car, the dataset is generated, and the generator *is* the source: each layout comes from a seed, and the observation is cast against that layout's exact wall geometry at run time. Nothing is stored in a file.

3.1.1 Layout generator

In `tinyml_racing/sim/track/`: random points in an ellipse, convex hull, vertices pulled inward so the lap turns both ways, the outline slid apart to make a main straight, hooks and chicanes added to the longer edges, then every vertex rounded into an arc. Shape targets come from the 18 F1 circuits in the TUMFTM racetrack-database [6]: lap 3.0–6.0 km, 8–13 corners, longest straight 12–17 % of the lap, corridor width 13 m within 10 %. The minimum corner radius is 2× the car's full-lock radius (17.13 m) from `CarParams`, so the generator cannot draw a corner the car cannot take. Walls are kept as arcs and straights and offset in closed form, so curvature is exact everywhere.

3.1.2 Label source

A minimum-curvature racing line, solved per layout as a bound-constrained sparse least-squares problem over one lateral offset per centerline sample (`scipy.optimize.lsq_linear`, bounds 1 m inside the corridor edge). A pure-pursuit controller follows that line using privileged state (full pose, exact path curvature) and a speed plan that brakes for the tightest corner within stopping distance. It never reads the LiDAR; its commands are the labels.

3.1.3 Sensor model

In `sim/lidar.py`: 28 rays over 240°, 150 m range. The bearings are warped by $\phi(u) = \phi_{\max} \tan(au)/\tan(a)$, with u evenly spaced on $[-1, 1]$ and $a = \arccos(f^{-1/2})$, so that rays land at even distances down a corridor instead of at even angles. The one knob is f, `ray_focus`, the ratio of edge to centre spacing ($f = \sec^2 a$, so $f = 1$ is a uniform fan). At $f = 9$ that puts 10 of the 28 beams inside ±20° at 3.9° spacing, instead of 4 beams at 8.9°. Only those centre beams see far enough ahead to spot a corner in time to brake. Ranges are divided by `max_range`, so 1.0 means "nothing in range".

3.2 Features, labels and augmentation

Table 2: The four input blocks and how each one is scaled before it reaches the network.

block	width	contents	normalizer
scan	28	current sweep	divide by 150 m
scan history	28	$\text{scan}_t - \text{scan}_{t-1}$	already normalized
car state	4	v_x, v_y, r, δ	top speed, 12.5 m/s, 1.3 rad/s, 0.30 rad
throttle history	1	previous pedal command	already in $[-1, 1]$

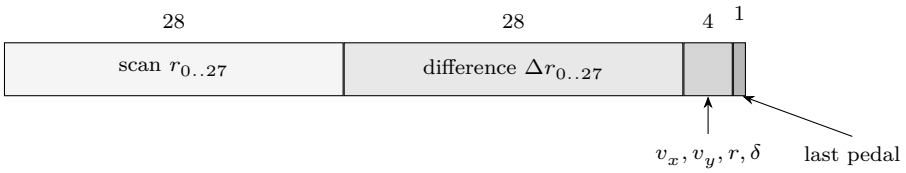

Figure 1: Layout of the 61-input observation vector: 28 normalized ranges, the 28 first differences between this sweep and the last one, four car-state channels and the previous pedal command.

That is 61 inputs (Figure 1). The number is computed from the block widths rather than hard-coded, and a test checks every block combination against the advertised space.

3.2.1 Augmentation

Both mechanisms are physical. The sensor adds Gaussian noise at $\sigma = 0.002$ of full scale (0.3 m at 150 m) and drops 1 % of beams. Beam dropout is what earns the difference block its 28 inputs: the policy rides over a dead centre beam instead of steering at it. The cloning dataset also uses DART

noise [3] — $\mathcal{N}(0, 0.05)$ on the *executed* action, with the label still the expert's — so the states are ones a slightly worse driver reaches and the labels show the way back.

3.2.2 Whitening

The blocks arrive normalized, but they are not centred and their variances differ: over 4,000 expert steps, per-feature standard deviations range from 0.06 to 0.57. `VecNormalize(norm_obs=True)` [7] sits on top; it is folded into layer 0 at export, so the board pays nothing for it (Section 5.2.1). Removing it end to end wrecks PPO: it trains to 634 ± 173 reward instead of $2{,}519 \pm 644$ at a 400k-step budget.

3.3 Splits and leakage

`TRAIN_SEED_RANGE` is $[0, 2\times10^9)$ and `EVAL_SEED_RANGE` is $[2\times10^9, 2^{31})$. A layout is fully determined by its seed, so disjoint seed ranges mean disjoint tracks.

Table 3: Data splits. The seed ranges do not overlap, so training and evaluation tracks do not either, and the last two rows come from streams that cannot collide.

split	seeds	size	used for
training layouts	train range	1,024 (16 workers, 64 each)	PPO rollouts
cloning / distillation	train range, separate RNG streams	200,000 samples each	supervised stages
supervised validation	inside the cloning set	5 % of rows, whole episodes	train/val MSE
selection layouts	eval range, `default_rng(0)`	5 layouts, then 8	choosing `best.pt`, every comparison
test layouts	eval range, separate stream	16 layouts	one final score, nothing else

Three guards against leakage. Supervised validation is held out by whole *episode*, because a random row split on a 50 Hz trajectory would put 20 ms neighbours on both sides. The normalizer is fit on training rows only. The PPO worker seed base is `SeedSequence([seed, 0x50504F])`, so worker 0 cannot replay the cloning layouts, and distillation shifts it again.

The last two rows matter for how the results should be read. Selection layouts come from a fixed `default_rng(0)` over the eval range, so every configuration in this report is compared on the same eight layouts with the same starting positions — but those layouts also chose `best.pt`, so they measure *this* model rather than generalization. The test layouts were drawn after the model was frozen and scored once (Section 6.4).

3.3.1 No onboard sensor data

The board has no LiDAR attached. Nothing was collected from its own sensors and nothing was fine-tuned on device; its job is inference only.

3.4 Software and hardware

Python 3.14, PyTorch, Stable-Baselines3 [7], Gymnasium [8], NumPy, SciPy, numba (physics kernel), ONNX + onnxruntime, pygame (viewer), pyserial (link). The device build uses `arduino-cli` with the `arduino:mbed_nano` core, compiled by the system `arm-none-eabi-g++` 16.2 instead of the core's gcc 7.2, which cannot compile the ACLE DSP intrinsics [9]. Training ran on CPU: a GPU round trip costs more than a 61-to-2 forward pass.

4 Model Development and Advanced Components

4.1 Models evaluated

Table 4: The four drivers we compared. Only the last one is deployed.

approach	sees	shape	parameters
pure-pursuit expert	privileged state	analytic	—
cloned expert	61-float observation	61-64-64-2	8,450
PPO policy	61-float observation	61-64-64-2	8,450
distilled student (deployed)	61-float observation	**61-16-8-2**	**1,146**

tanh everywhere, with a linear head clipped to $[-1, 1]$. A regression head on a small dense trunk is what the course guidance recommends for this kind of task [10]. Two hidden layers rather than one because Thomas et al. compare one against two node by node over ten function-approximation datasets, and two wins in nine [5]. Width was set by the flash budget. The critic is 256-256 and is thrown away at export, so its units cost the device nothing.

4.2 Why the proposal's plan changed

The proposal was pure imitation: a 1D CNN teacher distilled into a small student. The clone works as a regressor and fails as a driver — train and validation loss within 1.2× of each other, yet only 0.15 laps and a crash on every evaluation layout. More data would not obviously help, since the clone already fits the data it has; what we can say from these runs is that the limitation is a mismatch between imitating a privileged expert and the observation the student actually gets. Pure pursuit reads a map, the student reads 28 ranges, and no amount of label-matching closes that gap. We did not run the ablation that would separate capacity from data directly, so this stays a reading of the evidence rather than a measured claim. Either way the teacher had to change: PPO [1] trains the small net to lap instead of to imitate, and the clone is not wasted — it is PPO's warm start.

4.3 Training details

4.3.1 Stage 1 — cloning

200,000 transitions from the expert with DART noise $\sigma = 0.05$ [3]. Loss is $\mathrm{MSE}(\mu_\theta, a_{\text{expert}}) + 0.5\,\mathrm{MSE}(V_\theta, \hat{V})$ against the deterministic mean, never a sample. Adam, lr 10^{-3}, 100 epochs, batch 512, 5 % episode-wise validation, no early stopping, `log_std` frozen. A second noise-free pass supplies the critic's return targets, because the noisy pass's rewards belong to the executed action rather than the labelled one.

4.3.2 Stage 2 — PPO

PPO [1] via Stable-Baselines3 [7]: 16 environments, 2,048-step rollouts (32,768 steps per update), Adam (SB3's default) at lr 3×10^{-5}, batch 256, clip 0.2, entropy 0.003. The credit window is set as a *duration*, so it means the same thing at any control rate: $\gamma = e^{-\Delta t/\tau_\gamma} = 0.99800$ and $\lambda = 1 - \Delta t/\tau_\lambda = 0.9933$, with $\tau_\gamma = 10$ s, $\tau_\lambda = 3$ s and $\Delta t = 20$ ms. Three seconds because the manoeuvre credit has to cross is a corner setup, and the racing line's swing from outside to apex and back takes 1.4–1.6 s at racing speed; the earlier 1 s setting decayed the apex reward before credit reached the decision to run wide.

The warm start carries over three things, and dropping any one breaks the run: the `VecNormalize` statistics, the fitted critic (a good actor next to a random critic gives advantages that mean nothing), and a smaller starting exploration std, $e^{-1} = 0.368$, because SB3's default of 1.0 over a 2-wide action range throws the clone away. The run reached 7,340,048 steps; `best.pt` was taken at 7,100,000, by mean deterministic return over five fixed selection episodes, before a late dip.

4.3.3 Reward

Dense progress along the racing line (Δs over the distance top speed covers in one step), minus $0.02\times$ the normalized change in steering rate, plus potential-based shaping on distance to the line with weight 5, minus 250 for leaving the track. The potential is forced to zero at the terminal state, which makes the telescoping sum cancel and leaves the optimal policy unchanged [2]. Episodes end when the car leaves the track, at 200 s, or after covering less than 1 % of top speed for 5 s. The last rule exists because standing still pays zero, so a parked car would otherwise burn the whole budget.

4.3.4 Stage 3 — distillation

200,000 fresh samples labelled by the deterministic mean of `best.pt`, same optimizer and schedule as stage 1 (Adam, lr 10^{-3}, 100 epochs, batch 512) and the same DART noise, fit into 61-16-8-2. This did more than shrink the net: it removed the PPO policy's 40 % crash rate, because the exploration noise the stochastic policy carries is what was putting it into walls.

4.4 Advanced components

Three of them, matching items 1, 2 and 4 of the course list. Item 3, fine-tuning on onboard sensor data, does not apply here: the board has no LiDAR to collect from.

4.4.1 Cascaded compression, measured stage by stage

Three reductions in series, each measured on its own [10]. Figure 2 is the chain itself.

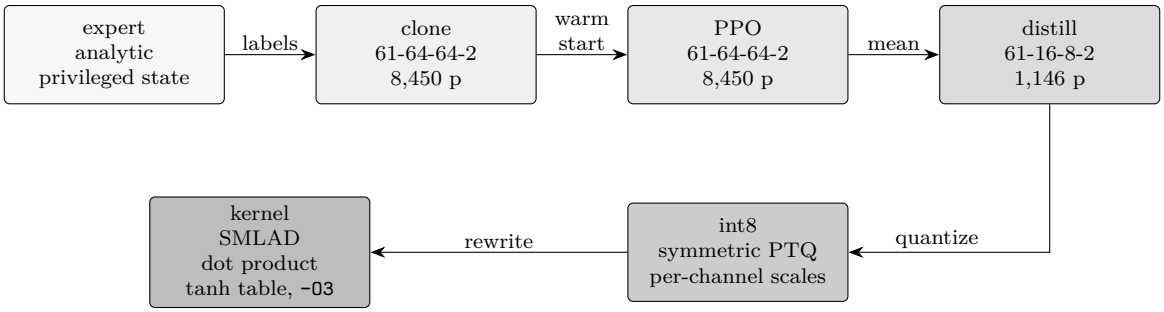

Figure 2: The compression chain. Distillation changes the architecture, int8 changes the numeric format, and the kernel rewrite changes neither — it runs the same arithmetic faster. The clone and the PPO policy share one architecture and differ only in how they were trained.

Section 6 measures each step, including the configurations we rejected.

4.4.2 Multiple models on identical layouts

Seven drivers, same seeds and same starting positions: the analytic expert, the clone, the PPO policy, the distilled student, and that student as float32, as int8 on the host, and as int8 on the board. Cloning versus PPO is a supervised-versus-reward comparison, and its outcome is why the deployed model looks the way it does.

4.4.3 Output beyond the Serial Monitor

The board does not print to a Serial Monitor. It answers a binary framed protocol over USB-CDC, and a live pygame visualizer drives a car around a rendered track from those answers in real time. Section 5.4 specifies the frames and the two host-side consumers; Figure 5 is the visualizer with the board in the loop.

4.5 Why this configuration was deployed

The 16-8 student is the only candidate that is both the best driver and small enough: 5,561 reward against its own PPO teacher's 4,490, with 7.4× fewer parameters. int8 over float32 halves the constants for 0.6 % of reward. Neither QAT nor pruning was applied; Section 6.2.1 has the measurements behind both refusals.

5 TinyML System Design and On-Device Deployment

5.1 System diagram

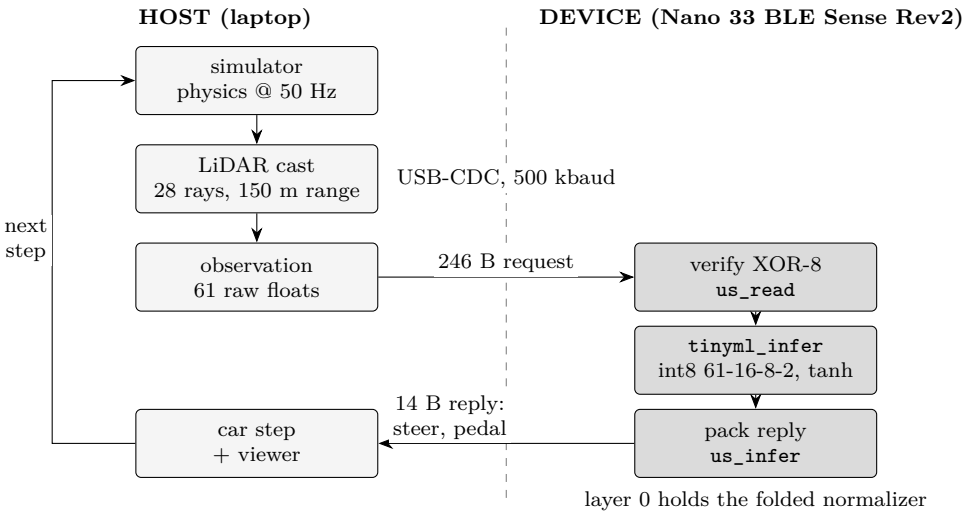

Figure 3: One 20 ms control step. Everything right of the dashed line runs on the flashed board, which is the only thing that evaluates the network.

5.2 Conversion and integration

We did not use TensorFlow Lite Micro. Its interpreter, resolver and tensor arena cost several kilobytes no matter what, which is a lot for 1,384 bytes of weights, and its int8 kernels do not expose the arithmetic we need to stay bit-exact against a host reference. The path is direct instead. `export.py` pulls the actor out of the SB3 snapshot, refuses any trunk that is not `Tanh`, folds the `VecNormalize` affine into layer 0, and records 2,048 calibration states plus 256 reference pairs from the policy's own rollouts. `quantize.py` does symmetric int8 with no zero points, following Krishnamoorthi [4]: weight scales per output channel ($\max|W|/127$ per column), activation scales per tensor, and dequantization as one multiply on the int32 accumulator. Per channel is necessary because once normalization is folded in, layer 0's columns differ by an order of magnitude in norm and one shared scale would quantize the small ones to zero. `codegen.py` writes `model.h` — int8 blobs in output-major order so the inner loop is contiguous, per-channel multipliers and biases, and a CRC32 of the whole model as `MODEL_DIGEST` — and `onnx_export.py` writes `actor.onnx` by hand as a portable record, not the deployment path. `tinyml.h` then reimplements the quantized model operation for operation, and the only operators it needs are dense matrix-vector, tanh and clip.

5.2.1 Preprocessing consistency

There is no preprocessing on the device: normalization is folded into layer 0, so the board gets the raw observation. Three checks enforce this. The build fails if ONNX disagrees with the float32 actor by more than 10^{-3} on the reference vectors (observed 6.5×10^{-7}). The board's identity string carries `MODEL_DIGEST`, arch, activation and widths, which the host re-derives from `actor.npz` before it will drive, so a stale flash is a refusal rather than a wrong answer. And the toolchain is validated by compiling the DSP intrinsics, not by reading a version number.

5.3 Device constraints and how they were met

Table 5: The device budget against what the model and the whole sketch actually use. The round trip is USB and host scheduling, not inference; it varies by a tenth of a millisecond between runs.

resource	budget	model	whole sketch
flash	983,040 B	2,412 B of constants (0.25 %)	104,684 B (10 %)
RAM	262,144 B	549 B scratch + 244 B input (0.30 %)	44,864 B (17 %)
time	20,000 µs per step	138 µs mean, 161 µs worst (0.7 %)	5.8–5.9 ms round trip

Scratch is three static buffers of `MODEL_MAX_WIDTH` — two float ping-pong, one int8 — sized from the widest layer, so it does not grow with depth. The kernel is not reentrant, which is stated in its contract; the sketch is single-threaded.

Speed came from four build decisions, not from changing the model. Table 6 is the chain in the order we found it.

Table 6: Getting the kernel under budget, 8.3× in four steps with the network untouched. The first three rows were measured on an earlier 64-32-16-2 net, so they compare with each other rather than with the last two. Only the tanh table changes output bits, and it is bounded below the int8 LSB.

step	inference	cost to exactness
mbed core's `-Os`, scalar dot product	1,150 µs	none
`-O3 -funroll-loops` on the sketch	693 µs	none
`SMLAD` + `SXTB16` dot product	523 µs	none
`tanhf` to a 257-knot table	234 µs	one int8 level, bounded
`lrintf` to the `VCVTR` instruction	138 µs	none

Two of the steps are the same finding twice: the toolchain declining to use an instruction the chip has. GCC emits zero `SMLAD` here because its vectorizer targets NEON/MVE rather than the M4's packed-halfword GPR ops, so the dot product widens four int8 bytes with `SXTB16` and accumulates them with two `SMLAD`s by hand [9]. And `lrintf` is a library call at every optimization level, `-ffast-math` included, which also made the code larger; rounding is now the `VCVTR` instruction libm was being called for, with the NaN and saturation guards moved after the conversion where they are integer compares. The tanh table is 257 knots at 1/32 spacing over $[0, 8]$, worst interpolation error 9.4×10^{-5}, 84× below the 1/127 LSB it gets requantized to one line later.

5.4 Output method

The link is a binary framed protocol at 500 kbaud. Two frames carry a control step, `'R'` out and `'A'` back, laid out in Figure 4. Two more handle everything else: the board answers a bad or missing checksum with `'E'` plus a uint16 byte count and a named sentinel, and answers `'?'` with an identity string carrying its arch, activation, widths and model digest.

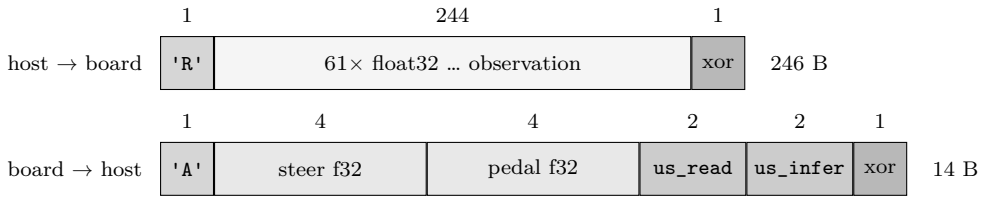

Figure 4: Byte layout of the two frames that carry a control step, with each field's size in bytes above it. The reply is drawn to scale at 7 mm per byte; the request's 244-byte payload is compressed to fit. Both frames end in an XOR-8 checksum over everything before it.

Requests go out in 64-byte USB packets with a 50 µs gap, so the device's ring buffer is never overrun. `us_read` times the request through checksum verification and `us_infer` times `tinyml_infer` alone, so the reported latency is the kernel and not the wire. A rejected frame drains to an idle gap first, so the host's next command byte survives.

The consumer is the pygame viewer (`tinyml-watch`), shown in Figure 5. `tinyml-board` is the non-graphical one: it replays the reference frames and diffs the board against the host emulator. The run below is one of four; Section 6.3 has the spread.

```
$ uv run tinyml-board --n 256
frames                256
vs emulator (max)     5.960e-08
inference             139 us mean, 180 us max
round trip            5.80 ms mean, 5.93 ms max
```

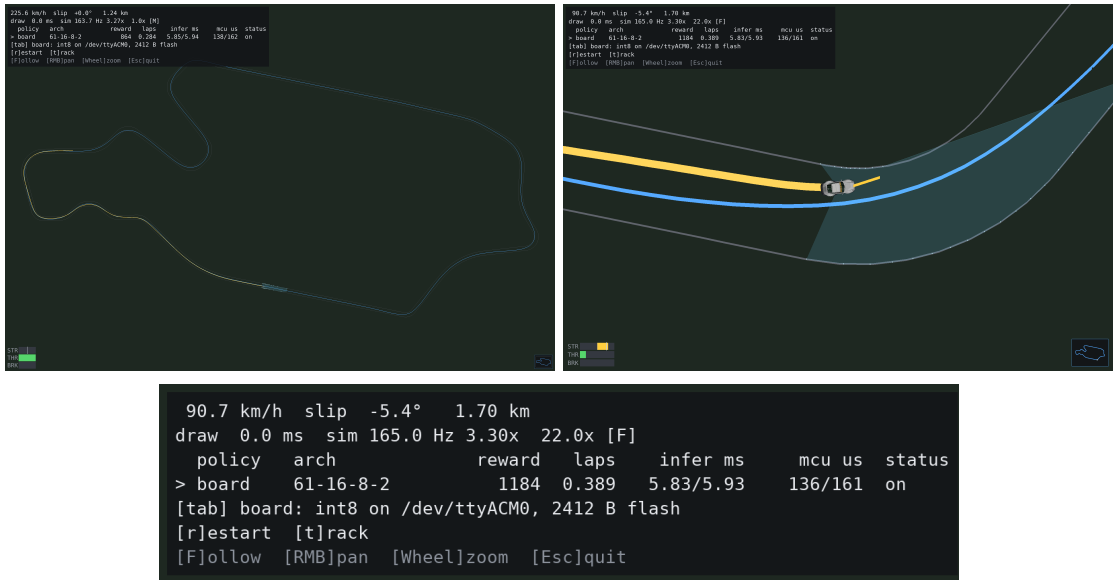

Figure 5: On-device inference on held-out layout seed 2125453163. The flashed Nano is the only thing choosing steering and throttle. Top left: the whole layout, with the yellow trace showing the last 900 m of driven path against the blue racing line. Top right: the same episode 400 steps later, in a corner, with the LiDAR fan drawn and the steering at lock. Bottom: that frame's HUD enlarged. `mcu us 136/161` is the mean and worst `us_infer` reported by the device itself over the last 60 steps, next to the `5.83/5.93` ms USB round trip.

6 Results and Analysis

6.1 Baseline performance before compression

Two views, because the supervised metric ranks models it cannot actually judge. Table 7 is the supervised one.

Table 7: (A) The two supervised stages: 200,000 samples, 5 % held out by whole episode.

stage	architecture	train MSE	val MSE	val/train	interpretation
clone	61-64-64-2	0.00146	0.00172	1.18×	Fits the expert. Cannot drive.
distill	61-16-8-2	0.00876	0.00889	1.01×	Worse per step. Deployed anyway.

Neither overfits, so the gap is not fitting but driving. Figure 6 scores the same models both ways, and the ranking flips: the model with the worst validation MSE drives best, and the one with the best MSE cannot finish a corner. That is why every later decision is judged in closed loop.

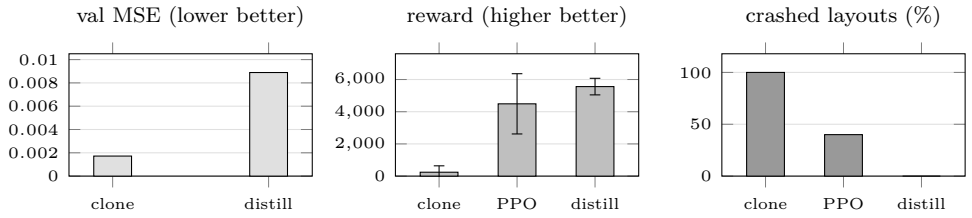

Figure 6: The same three policies under a supervised metric and under driving, over five selection layouts with deterministic actions and a clean sensor. The clone (8,450 parameters) takes 242 ± 397 reward at 0.15 laps and crashes on every layout; PPO (8,450) takes $4{,}490 \pm 1{,}871$ at 1.53 laps and 40 %; the deployed 1,146-parameter student takes $5{,}561 \pm 515$ at 1.77 laps and crashes on none. PPO has no supervised panel because it is not fit to labels. Whiskers are one standard deviation across layouts.

Re-scored on the final protocol of eight selection layouts of 10,000 steps each, the selected baseline gives $5{,}519 \pm 470$ **reward, 1.749 laps, 0 % crashes, 231.6 km/h top speed**.

6.2 Compressed model and TinyML tradeoffs

Table 8: (B) Float32 against int8. Eight selection layouts, 10,000 steps each, clean sensor, identical seeds and starting positions; action MAE is over the 256 reference frames against the PyTorch policy. Both int8 rows share one scheme, specified in Section 5.2: symmetric post-training int8, `QMAX = 127`, no zero points, per-channel weight scales, per-tensor activation scales, 2,048 calibration states, plus the 257-knot tanh table [4]. The board row adds the SMLAD dot product and the VCVTR quantizer at `-O3 -funroll-loops`. The float32 baseline was never flashed, so it has no RAM or latency figure. The bold row is the deployed configuration.

configuration	reward	Δ	laps	crash	MAE	flash	RAM	latency
float32 baseline	5,519	—	1.749	0 %	—	4,584 B	—	—

configuration	reward	Δ	laps	crash	MAE	flash	RAM	latency
int8, emulator	5,483	−0.65 %	1.740	0 %	0.0170	2,412 B	549 B	—
int8, board	5,486	−0.60 %	1.740	0 %	0.0170	2,412 B	549 B	138 µs

Both int8 rows are 1.90× smaller than float32, which is not the whole story.

6.2.1 Rejected alternatives, measured

Three ways to cut latency were flashed and replayed before being refused, and Figure 7 is what they isolate: tanh costs 22 µs, the float dequantization 7 µs, the rounding and rails 6 µs, leaving 103 µs of MACs and loads that no shortcut touches. All three break exact equality against the emulator for a sixth of a kernel that already fits in 0.7 % of the control interval. QAT was skipped because post-training int8 already costs only 0.6 % of reward at a 0 % crash rate; pruning because at 1,120 weights a sparse format's indices cost more flash than the zeros save.

6.2.2 Reading the compression ratio

1.90× is deliberately pessimistic. Figure 7 shows why: int8 is charged for the 1,028-byte tanh table, while float32 is not charged for libm's `tanhf`, which it would also need. On weights alone the ratio is 3.31× (4,584 → 1,384 B). The table is a fixed cost, so the smaller the network, the worse int8 looks — a real and slightly counterintuitive TinyML tradeoff. The right panel is the other side of the same trade: the table bought back more time than it cost.

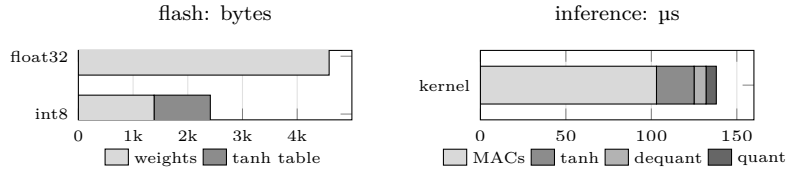

Figure 7: Left: the deployed 2,412 bytes against the float32 baseline's 4,584. The tanh table is 43 % of what int8 flashes, which is what turns 3.31× on weights into 1.90× deployed. Right: that same table costs 22 µs of the 138 µs inference, and the multiply-accumulate work no shortcut can remove is 103 µs. Both panels plot numbers from Table 8 and Section 6.2.1.

6.3 On-device evaluation

Every action in the closed-loop row of Table 9 was computed on the Arduino; the host only stepped the physics.

Table 9: (C) On-device evaluation. 256 samples for each of the two regression outputs, well past the 10–15 minimum. The 256- and 128-frame rows replay the held-out reference set; the closed-loop row is eight layouts of 10,000 board-driven steps, −0.60 % against float32.

output	samples	metric vs float32	worst	notes
steer	256 frames	MAE 0.0290	0.0969	4.8 % of the 2-wide range
throttle / brake	256 frames	MAE 0.00496	0.0969	5.8× smaller than steer
both channels	256 frames	MAE 0.0170	0.0969	the "int8 action error"

output	samples	metric vs float32	worst	notes
vs emulator	128 frames	max 5.96×10^{-8}	—	bit-exact; wire rounding
closed loop	80,000 steps	5,486 ± 469	—	1.740 laps, 0 crashes
latency	128 frames	138 µs mean	161 µs	0.7 % of the 20 ms step

The board-versus-emulator row is a *correctness* check — the only measurement that catches `tinyml.h` and `quantize.py` drifting apart — and at 5.96×10^{-8} the deployed function is the one we evaluated on the host. The int8-versus-float32 rows are the *cost* of quantization. Over 80,000 steps it adds up to a 0.05 % reward gap between board and emulator, with the board slightly ahead rather than behind, so it is noise rather than bias. Replaying four times moves the latency across 136–141 µs mean and 161–180 µs max while the emulator diff and `MODEL_DIGEST` stay identical, so that spread is instruction scheduling, not a different model.

6.4 A test set that selection never saw

Everything above is scored on the eight layouts `default_rng(0)` draws — the stream that also picked `best.pt`. They are held out from *training*, not from *selection*, so they measure this model rather than generalization. With the model frozen (`MODEL_DIGEST 0x7a3977b9`) we drew 16 fresh layouts from the same range on a stream selection never touched, checked them against the first 64 seeds that stream can produce, and scored all three configurations once. Nothing below fed back into a decision.

Table 10: (E) The frozen model on 16 layouts selection never saw, 10,000 steps each, clean sensor. Intervals are 95 % t intervals over tracks, $n = 16$. "Lapped" counts layouts where the configuration completed at least one lap, which is stricter than not crashing: a policy can run out of budget short of the line without leaving the track.

configuration	reward, 95 % CI	laps	lapped	crashed
float32	5,301 [4,856–5,747]	1.546	15/16	0/16
int8 emulator	5,261 [4,698–5,824]	1.533	15/16	1/16
int8 board	5,062 [4,432–5,692]	1.480	14/16	1/16

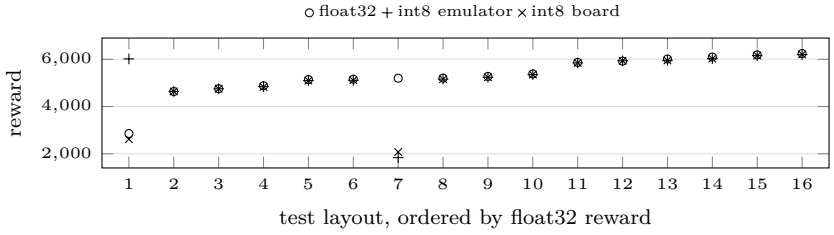

Figure 8: Per-track reward on the 16 test layouts. The three configurations coincide on the 14 layouts where none of them leaves the track. Layouts 1 and 7 are marginal corners: at 1 the board leaves the track and float32 survives but only reaches 0.76 laps, and at 7 the emulator leaves it while the board stops at 0.61 laps without leaving and float32 laps normally. Aggregate means hide all of this.

On those 14 layouts the three agree closely — 5,483, 5,451 and 5,450 — and the paired board-minus-float32 difference is -33 ± 14 reward, 0.6 %: the selection set's quantization cost, now with an interval. Over all 16 the board is 4.5 % behind, but a paired t test cannot separate them ($p = 0.23$) because layouts 1 and 7 dominate the variance, while a Wilcoxon signed-rank test does ($p < 0.001$) because the board is behind on 14 of the 16. That is the honest summary: quantization costs a little almost everywhere, consistently enough to detect, and at a marginal corner it can cost the lap.

6.5 Ten representative on-device frames

Table 11: (D) Ten frames across the steering range, from the 256-frame held-out reference set the board replays. "True" is the float32 policy on the same observation; "int8" is what the board computes, bit-identical to it within 5.96×10^{-8}.

#	frame	steer true	steer int8	err	pedal true	pedal int8	err
1	1	−0.7915	−0.7638	0.0278	+1.0000	+1.0000	0.0000
2	18	+0.3099	+0.3367	0.0267	+0.9969	+0.9956	0.0014
3	62	−0.0743	−0.0673	0.0070	+0.9919	+0.9912	0.0007
4	141	+0.1215	+0.1619	0.0404	+0.9983	+0.9994	0.0011
5	153	+0.0079	+0.0777	0.0698	+1.0000	+0.9977	0.0023
6	159	+0.0512	+0.0488	0.0024	+0.9948	+0.9938	0.0011
7	190	−0.3021	−0.2651	0.0370	+1.0000	+1.0000	0.0000
8	211	−0.1351	−0.1051	0.0300	+0.9084	+0.8848	0.0236
9	224	−0.0266	+0.0064	0.0330	−0.0324	−0.0205	0.0119
10	240	+1.0000	+1.0000	0.0000	−0.3509	−0.3501	0.0008

Evidence: `reference_in/reference_out` in `artifacts/actor.npz` beside `report.json`; `manifest.json` records `board_verified: true` and digest `0x7a3977b9`. Reproduced by `uv run tinyml-board`.

Rows 1 and 10 are the interesting ones. At full lock (10) the error is exactly zero, because the head clips to ± 1 and quantization cannot move a saturated output. At hard opposite lock (1) it is 0.0278 — 0.4° of a 17° rack. The worst case in the set, 0.0969, happens near zero steering (row 5 is closest at 0.0698), which is the harmless place for it: on a straight, with the steering-rate penalty smoothing the command, 0.07 of steering for one 20 ms step moves the car sideways by millimetres.

6.6 Analysis

6.6.1 Which tradeoffs were actually hard

Size against accuracy was not one of them: int8 saves half the flash and costs 0.6 % of reward on every layout the policy laps, so there was nothing to weigh. Three others were real.

Capacity against drivability. The model with the worst per-step error drives best, because PPO can find a policy that fits in 1,146 parameters, while cloning can only approximate one that reads a map.

Exactness against speed. Full-integer requantization would save 35 µs of the 138. We turned it down because it replaces an equality test with a tolerance and leaves two definitions of the model in the codebase.

The tanh table against the compression ratio: a fixed 1,028 bytes that the network cannot amortize, worked through in Section 6.2.2.

6.6.2 Observed failure cases

The clone crashes on every selection layout at 0.0017 validation MSE. That is the headline failure and the reason the project changed shape. PPO crashes on 40 % of them, fixed by distilling the deterministic mean rather than by improving the network. The deployed configuration crashes on none of the eight, so on that set the residual error is a cross-track offset rather than a failure: a median 1.22 m from the racing line on a 6.5 m corridor half-width. The test set corrects that picture — one layout in 16 puts the board off the track and a second leaves it short of a lap, both at corners where float32 is also marginal, so the deployed failure mode is not "never crashes" but "crashes rarely, on corners the policy is borderline on anyway". Quantization error concentrates in steering (5.8× the throttle MAE), the channel the closed loop is most sensitive to, and the worst frames are near zero steering rather than at lock.

7 Challenges, Limitations, and Future Work

7.1 Deciding what to trust

The first version optimized a metric that could not see the failure — a good validation MSE beside a crash on every layout, as Section 6.6.2 lays out. The fix was closed-loop gates (laps and reward on held-out layouts, identical seeds across configurations) and exact equality against a host reference instead of a tolerance on a loss.

7.2 The speed work was a build problem, not a model problem

Table 6 is the whole story: 8.3× with the network untouched, and the two compiler decisions between them beating the hand-written SIMD that followed. We had expected the SIMD to be the win and the flags to be housekeeping. The lesson is that on an M4 the build deserves the same scrutiny as the architecture, and that "the compiler will do it" is worth checking against the disassembly.

7.3 What partially worked

The warped fan is justified by ray budget — 10 useful beams inside ±20° instead of 4 at the same cost — but cloning under five fan geometries leaves steering error flat, and a two-seed PPO A/B could not separate them, so we cannot claim a lap-time win. The `--scan-history 1` versus `2` comparison on the noisy sensor was never run, so the 28 difference inputs rest on the dropout argument rather than on a measurement.

7.4 Limitations

The LiDAR is simulated, so we make no sim-to-real claim: real time-of-flight returns have multipath, intensity-dependent dropout and beam divergence that Gaussian noise plus 1 % dropout does not model. Training is host-only; the board does inference, so there is no on-device learning. The link is USB, not radio, so the car cannot leave the desk. The test set of Section 6.4 fixes the split we were missing, but it is one pass over 16 layouts from the same generator, so it bounds this generator's distribution and nothing wider. Sixteen tracks also pins the crash rate very loosely: one failure in

16 is anywhere from 0.2 % to 30 % at 95 % confidence. And `tinyml-build` has no lap or reward gate: it refuses on ONNX parity, digest mismatch and toolchain, so a model that drove worse would still be recorded and flashed.

7.5 Next steps

Close the sim-to-real gap where it is cheapest: an RPLIDAR C1 on a bench, real sweeps against a known wall layout, and a measurement of whether the policy survives the actual noise distribution. After that, layer-0 channel blocking. The first layer reloads its 61-byte quantized input once per output channel, sixteen times over; doing two channels per pass halves that traffic and only reorders additions within separate accumulators, so it is the next speed-up that costs no exactness.

8 Reproducibility and References

8.1 Code structure

```
tinyml_racing/sim/     track generation, physics, LiDAR, pure-pursuit expert
tinyml_racing/ml/      env, reward, cloning, PPO, distillation, snapshots
tinyml_racing/deploy/  export, quantize, codegen, ONNX, evaluation, serial link
tinyml_racing/render/  pygame viewer and tinyml-watch
arduino/deploy/        deploy.ino, tinyml.h (kernel), link.h (protocol), model.h
tests/                 pytest suite, incl. tests/ckernel (C kernel vs emulator)
docs/findings/         the measurements behind each design decision
data/runs/<run>/       config.json, train.log, tb/, training/, artifacts/
```

8.2 Reproducing the deployed result

```
uv sync && arduino-cli core install arduino:mbed_nano
uv run tinyml-train --run-name myrun        # clone -> PPO -> distill
uv run tinyml-build --flash --port auto     # quantize, export, evaluate, compile, upload
uv run tinyml-board --n 256                 # board vs host emulator, frame by frame
uv run tinyml-watch --policy board --port /dev/ttyACM0   # the board driving, live
```

The deployed model is in the repository as `arduino/deploy/model.h` (`MODEL_ARCH "61-16-8-2"`, `MODEL_DIGEST 0x7a3977b9`), produced by run `run_20260814_172740` at git revision `999aaa0`. That run's `artifacts/` (`actor.npz`, `model.h`, `actor.onnx`, `report.json`, `manifest.json`) is the evidence behind Section 6 and is kept out of git along with the rest of `data/`. Every number in that section comes from `report.json`, `manifest.json` or `docs/findings/kernel-speed.md`, except the whole-sketch flash and RAM figures, which are `arduino-cli`'s size lines, and Figure 5, captured from the viewer with the flashed board in the loop.

8.3 Verification

`uv run pytest` runs 243 tests with no hardware and no display. `tests/ckernel/main.c` compiles the shipped `arduino/deploy/tinyml.h` with a host C compiler and diffs it against the NumPy emulator for exact equality, one case under ASAN, so the deployed kernel is checked without a board. The geometry test asserts that the sampled-polyline wall caster converges quadratically onto the exact one, which makes "exact" a measurement rather than a claim.

8.4 References

Cited above by number; PDFs of 1–5, 7 and 8 are in `docs/materials/`, and of 10 in `docs/materials/uw/`.

1. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. *Proximal Policy Optimization Algorithms.* arXiv:1707.06347, 2017.
2. Ng, A. Y., Harada, D., Russell, S. *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping.* ICML 1999.
3. Laskey, M., Lee, J., Fox, R., Dragan, A., Goldberg, K. *DART: Noise Injection for Robust Imitation Learning.* CoRL 2017, arXiv:1703.09327.
4. Krishnamoorthi, R. *Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper.* arXiv:1806.08342, 2018.
5. Thomas, A. J., Petridis, M., Walters, S. D., Malekshahi Gheytassi, S., Morgan, R. E. *Two Hidden Layers are Usually Better than One.* EANN 2017, 279–290. doi:10.1007/978-3-319-65172-9_24.
6. Heilmeier, A. et al. *TUMFTM/racetrack-database.* https://github.com/TUMFTM/racetrack-database, accessed August 2026.
7. Raffin, A. et al. *Stable-Baselines3: Reliable Reinforcement Learning Implementations.* JMLR 22(268), 2021.
8. Towers, M. et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments.* arXiv:2407.17032, 2024.
9. ARM Ltd. *ARM C Language Extensions (ACLE)* and *ARMv7-M Architecture Reference Manual.*
10. *Choosing DNN Architectures for TinyML Tasks* and *TinyML Compression Cookbook*, EE 446 course handouts.